

Algorithms

Lab 2 Solution

Gang Ferdinand Dinga

Question 1. Comparing Algorithms. Problem: Find the THIRD largest in an array.

Algorithm 1 : Idea – Use three loops one after another. First loop will find Max. Second loop will find Second Max, Third loop will find third max. Note that it is possible First max == second Max == Third Max as in 7, 20, 18, 4, 20, 19, 20, 3. and your program should return 20 in this case.

Solution:

Algorithm ALG1(arr[0..n-1])

Input array of integers

Output the third largest element in the array .

```
if n == 0 then error
max <- 0
for i <- 0 to n-1 do
  if arr[i] > max then
    max <- arr[i]
for i <- 0 to n-1 do
  if arr[i] > second and arr[i] <= max then
    second <- arr[i]
third <- 0
for i <- 0 to n-1 do
  if arr[i] > third and arr[i] <= second then
    third <- arr[i]
return third
```

Time complexity $O(3n) = O(n)$;

Algorithm ALG2(arr[0..n-1])

Input array of integers

Output the third largest element in the array .

```
if n = 0 then error
max <- arr[0]
preMax <- arr[0]

prePreMax <- arr[0]
```

```

for i <- 0 to n-1 do
  x <- arr[i]
  if x >= max:
    third <- second
    second <- max
    max <- x
  else if x >= second:
    third <- second
    second <- x
  else if x > third:
    third <- x
return third

```

Time complexity: single loop $O(n)$

Algorithm 3 — Ordered dictionary / sort-based approach

Two variants:

- (a) Insert all elements into an ordered dictionary (or multiset) that keeps elements sorted; then pick largest key (or top 3 values).
- (b) Simpler: sort the array descending and pick the first element (index 0) as the largest – but to follow the “third largest” semantics with duplicates you can just return element at index 0 after sorting descending and allow duplicates — or scan sorted list to find the third distinct value depending on exact requirement.

Time complexity:

- Building ordered dict (balanced BST or multiset): $O(n \log n)$ (insert each element $O(\log n)$).
- Sorting approach: $O(n \log n)$.

Empirical comparison (expected)

- ALG2 $\approx$ fastest (single linear pass, smallest constant).
- ALG1 $\approx$ linear but $\approx 3\times$ work of ALG2 but larger constant.
- ALG3 (ordered dict / sort) $\approx O(n \log n)$ so it will grow noticeably slower for large n (worse asymptotic).

So asymptotic ordering by running time for large n : $ALG2 \leq ALG1$ (both $O(n)$) $\ll$ $ALG3$ ($O(n \log n)$).

```
//first solution
public static int getThirdMax(int[] arr) { no usages
    int max = arr[0];
    int secondMax = max;
    int thirdMax = max;
    int indexOfMax = 0;
    int indexOfPreMax = 0;
    for(int i = 0; i < arr.length; i++)
    {
        if(arr[i] > max)
        {
            max = arr[i];
            indexOfMax = i;
        }
    }
    for(int i = 0; i < arr.length; i++)
    {
        if(arr[i] >= secondMax && i != indexOfMax)
        {
            secondMax = arr[i];
            indexOfPreMax = i;
        }
    }
    for(int i = 0; i < arr.length; i++)
    {
        if(arr[i] >= thirdMax && i != indexOfPreMax && i != indexOfMax)
        {
            thirdMax = arr[i];
        }
    }
    System.out.println("firstMax: " + max + " "
        + "secondMax: " + secondMax + " " + "thirdMax: " + thirdMax);
    return thirdMax;
}
```

```
public static int getThirdMax2(int[] arr) 1 usage
{
    int max = arr[0], preMax = arr[0], prePreMax = arr[0];
    for(int i = 0; i < arr.length; i++)
    {
        if(arr[i] > max)
        {
            prePreMax = preMax;
            preMax = max;
            max = arr[i];
        }
        else if(arr[i] > preMax && arr[i] <= max)
        {
            prePreMax = preMax;
            preMax = arr[i];
        }
        else if(arr[i] > prePreMax && arr[i] <= preMax)
        {
            prePreMax = arr[i];
        }
    }
    System.out.println("max: " + max + " " + "Pre Max: " + preMax + " Pre pre Max: " + prePreMax);
    return prePreMax;
}
```

	A	B	C	D
1	Feature	Algorithm 1 (Three Pass)	Algorithm 2 (Single Pass)	Algorithm 3 (Ordered Dict / Sort)
2	Description	Scans array 3 times — once each to find 1st, 2nd, and 3rd max	Uses one loop; maintains max, second, third in one pass	Builds a sorted structure or sorts array, then picks 3rd largest
3	Time Complexity	$\Theta(n)$ (3 passes $\times$ n steps each $\rightarrow$ still linear)	$\Theta(n)$ (1 pass)	$O(n \log n)$ (sort or ordered dict insertions)
4	Implementation Effort	Simple (but 3 loops)	Slightly more logic, 1 loop	Easiest if using built-in sort
5	Handles Duplicates			
6	Correctly?	Yes (using $\leq$ comparisons)	Yes (using $\geq$ comparisons)	Yes (if implemented as multiset or with equal values)
7	Best Case Runtime	$O(n)$	$O(n)$	$O(n \log n)$
8	Worst Case Runtime	$O(n)$	$O(n)$	$O(n \log n)$
9	Practical Speed	$\approx 3\times$ slower than Alg 2 (3 passes)	Fastest (one pass, smallest constant)	Slowest on large arrays (due to sort)
10				

Question2:

10, 1, n^3 , $n^{1/3}$, $\log(\log n)$, n^2 , $n^{1/2}$, $\log n$, $\log nn$, n^k ($k > 3$), $n^{1/k}$ ($k > 3$), $n \log n$, $\ln n$, $2n$, $3n$, nn , $n^{1/2} \log n$, $n^{1/3} \log n$, $n!$.

A	B	C	D
Rank	Function(s)	Θ -class	
1	10, 1	$\Theta(1)$	
2	$\log \log n$	$\Theta(\log \log n)$	
3	$\log n$, $\ln n$	$\Theta(\log n)$	
4	$n^{(1/k)}$ ($k > 3$)	$\Theta(n^{(1/k)})$	
5	$n^{(1/3)}$	$\Theta(n^{(1/3)})$	
6	$n^{(1/3)} \cdot \log n$	$\Theta(n^{(1/3)} \log n)$	
7	$n^{(1/2)}$	$\Theta(n^{(1/2)})$	
8	$n^{(1/2)} \cdot \log n$	$\Theta(n^{(1/2)} \log n)$	
9	n	$\Theta(n)$	
10	$n \log n$	$\Theta(n \log n)$	
11	n^2	$\Theta(n^2)$	
12	n^3	$\Theta(n^3)$	
13	n^k ($k > 3$)	$\Theta(n^k)$	
14	2^n	$\Theta(2^n)$	
15	3^n	$\Theta(3^n)$	
16	$(\log n)^n$ (interpreted from $\log n^n$)	$\Theta((\log n)^n)$	
17	$n!$	$\Theta(n!)$	
18	n^n	$\Theta(n^n)$	

Justification:

- **Constants (1,10) don't grow with $n \rightarrow$ smallest.**
- **$\log \log n \ll \log n$.**

- Any $\log n$ is smaller than any positive fractional power n^{ϵ} (so $\log n \ll n^{1/k}$ for any k).
- Among polynomial-root families, larger denominator $k \rightarrow$ smaller exponent; so $n^{1/k} \ (k > 3) < n^{1/3} < n^{1/2} < n < n^2 < n^3 < n^k \ (k > 3)$.
- Multiplying by slowly-growing $\log n$ increases a function slightly but does not jump it past the next polynomial power when exponents differ enough (hence $n^{1/3} \ll n^{1/3} \log n \ll n^{1/2}$).
- Exponentials with fixed base c^n outrun any polynomial as n grows ($2^n, 3^n, \dots$).
- $(\log n)^n = e^{n \ln \log n}$ eventually outruns c^n for fixed c , but it is still far smaller than $n! \approx (n/e)^n$ (Stirling: exponent $\sim n \ln n$) and much smaller than $n^n = e^{n \ln n}$. So ordering for large n is $2^n \ll 3^n \ll (\log n)^n \ll n! \ll n^n$.